

1. Student Attendance Analysis

A college maintains the daily attendance details of its students in the form of a list containing student IDs. Some students may have attended multiple sessions on the same day. The administration wants to identify the longest continuous sequence of sessions in which no student ID is repeated. Develop a solution that determines the maximum length of such a sequence.

Program

```
students = [1, 2, 3, 1, 4, 5]
seen = set()
left = 0
max_length = 0
for right in range(len(students)):
    while students[right] in seen:
        seen.remove(students[left])
        left += 1
    seen.add(students[right])
    max_length = max(max_length, right - left + 1)
print("Longest sequence:", max_length)
```

Output

main.pyVisualizeRun

```
1 students = [1, 2, 3, 1, 4, 5]
2 seen = set()
3 left = 0
4 max_length = 0
5 for right in range(len(students)):
6     while students[right] in seen:
7         seen.remove(students[left])
8         left += 1
9     seen.add(students[right])
10    max_length = max(max_length, right - left + 1)
11 print("Longest sequence:", max_length)
```

Output

Longest sequence: 5

2. Online Shopping Price Analysis

An online shopping application stores the prices of products viewed by a customer during a browsing session. The customer wants to identify a continuous range of products that provides the maximum possible total discount value. Given the discount values, determine the maximum value that can be obtained from any continuous range.

Program

```
discounts = [-2, 3, -1, 5, -6, 4]
current_sum = discounts[0]
maximum_sum = discounts[0]
for i in range(1, len(discounts)):
    current_sum = max(discounts[i], current_sum + discounts[i])
    maximum_sum = max(maximum_sum, current_sum)
print("Maximum discount value:", maximum_sum)
```

Output

main.py

Visualize

Run ▶

```
1 discounts = [-2, 3, -1, 5, -6, 4]
2 current_sum = discounts[0]
3 maximum_sum = discounts[0]
4 for i in range(1, len(discounts)):
5     current_sum = max(discounts[i], current_sum + discounts[i])
6     maximum_sum = max(maximum_sum, current_sum)
7 print("Maximum discount value:", maximum_sum)
```

Output

Maximum discount value: 7

3. Rainwater Collection System

A city installs buildings of different heights along a straight road. During rainfall, water gets collected between taller buildings. The engineering team needs to calculate the total amount of water that can remain trapped after heavy rainfall based on the heights of the buildings.

Program

```
heights = [3, 0, 2, 0, 4]
left = 0
right = len(heights) - 1
left_max = 0
right_max = 0
water = 0
while left < right:
    if heights[left] < heights[right]:
        if heights[left] >= left_max:
            left_max = heights[left]
        else:
            water += left_max - heights[left]
            left += 1
```

```

else:
    if heights[right] >= right_max:
        right_max = heights[right]
    else:
        water += right_max - heights[right]
    right -= 1
print("Trapped water:", water)

```

Output

main.py

Visualize

Run ▶

Output

```

1 heights = [3, 0, 2, 0, 4]
2 left = 0
3 right = len(heights) - 1
4 left_max = 0
5 right_max = 0
6 water = 0
7 while left < right:
8     if heights[left] < heights[right]:
9         if heights[left] >= left_max:
10             left_max = heights[left]
11         else:
12             water += left_max - heights[left]
13         left += 1
14     else:
15         if heights[right] >= right_max:
16             right_max = heights[right]
17         else:
18             water += right_max - heights[right]
19         right -= 1
20 print("Trapped water:", water)

```

Trapped water: 7

4. Employee Performance Analysis

A company stores the monthly performance scores of an employee for several months. The scores may contain both positive and negative values depending on the employee's performance. Management wants to identify the continuous period during which the employee achieved the highest overall performance.

Program

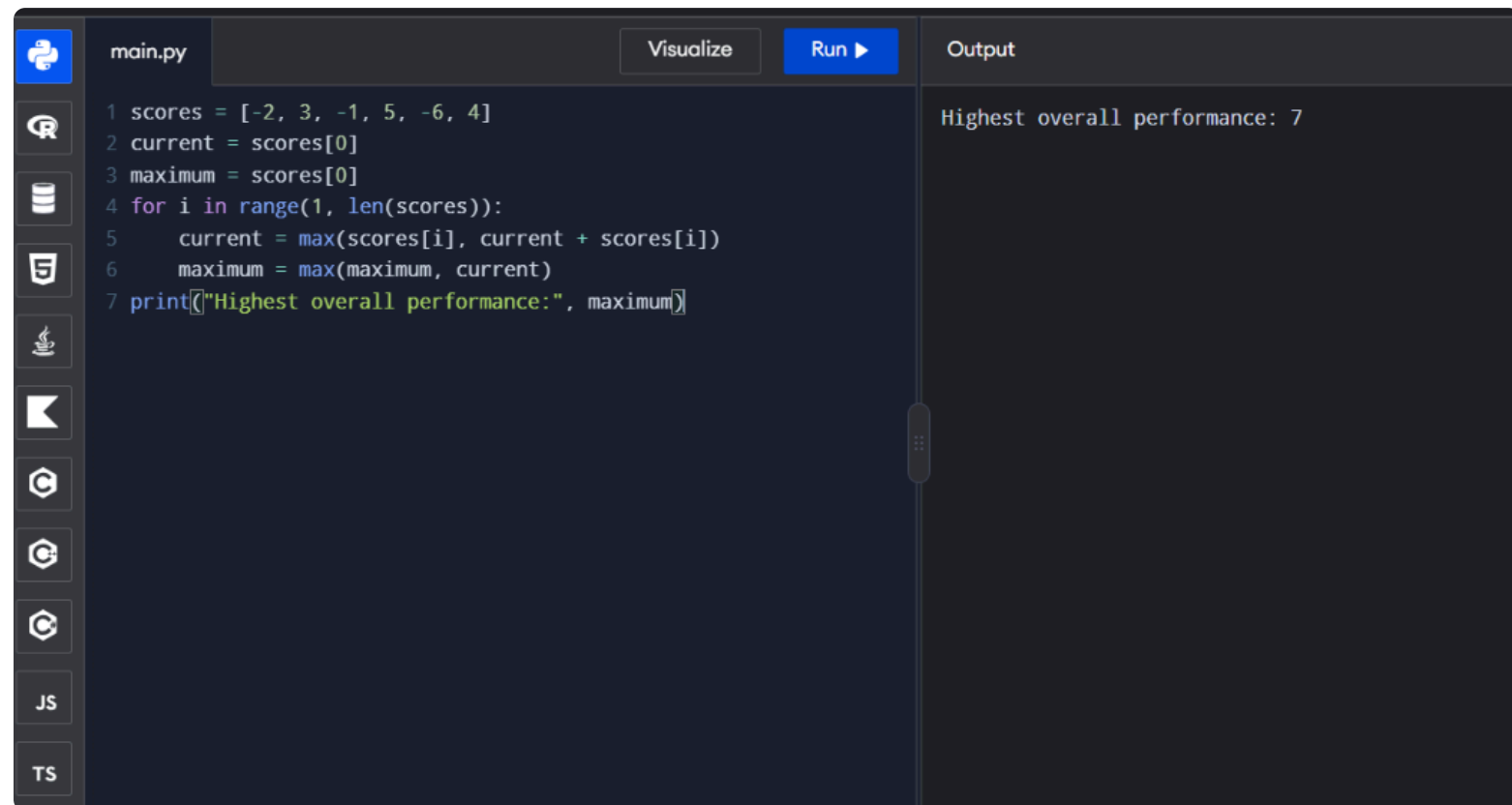
```

scores = [-2, 3, -1, 5, -6, 4]
current = scores[0]
maximum = scores[0]
for i in range(1, len(scores)):
    current = max(scores[i], current + scores[i])

```

```
maximum = max(maximum, current)
print("Highest overall performance:", maximum)
```

Output

A screenshot of a code editor interface. The editor has a dark theme. On the left, there's a sidebar with icons for Python, Jupyter, and other tools. The main area shows a file named 'main.py' with the following Python code:

```
1 scores = [-2, 3, -1, 5, -6, 4]
2 current = scores[0]
3 maximum = scores[0]
4 for i in range(1, len(scores)):
5     current = max(scores[i], current + scores[i])
6     maximum = max(maximum, current)
7 print("Highest overall performance:", maximum)
```

On the right, there's an 'Output' panel showing the result of running the code: 'Highest overall performance: 7'. Above the code editor, there are buttons for 'Visualize' and 'Run'.

5. Product Sales Analysis

A retail company stores the daily sales quantity of a product for several consecutive days. Due to seasonal changes, some days may have negative adjustments. The company wants to identify the period that produced the highest multiplication of sales-related values. Develop a solution to determine this maximum product.

Program

```
sales = [2, 3, -2, 4]
maximum = sales[0]
minimum = sales[0]
answer = sales[0]
for i in range(1, len(sales)):
    value = sales[i]
    # Negative numbers can change minimum into maximum
    if value < 0:
        maximum, minimum = minimum, maximum
    maximum = max(value, maximum * value)
    minimum = min(value, minimum * value)
    answer = max(answer, maximum)
print("Maximum product:", answer)
```

Output



main.py

Visualize

Run ▶

```
1 sales = [2, 3, -2, 4]
2 maximum = sales[0]
3 minimum = sales[0]
4 answer = sales[0]
5 for i in range(1, len(sales)):
6     value = sales[i]
7     if value < 0:
8         maximum, minimum = minimum, maximum
9     maximum = max(value, maximum * value)
10    minimum = min(value, minimum * value)
11    answer = max(answer, maximum)
12 print("Maximum product:", answer)
```

Output

Maximum product: 6

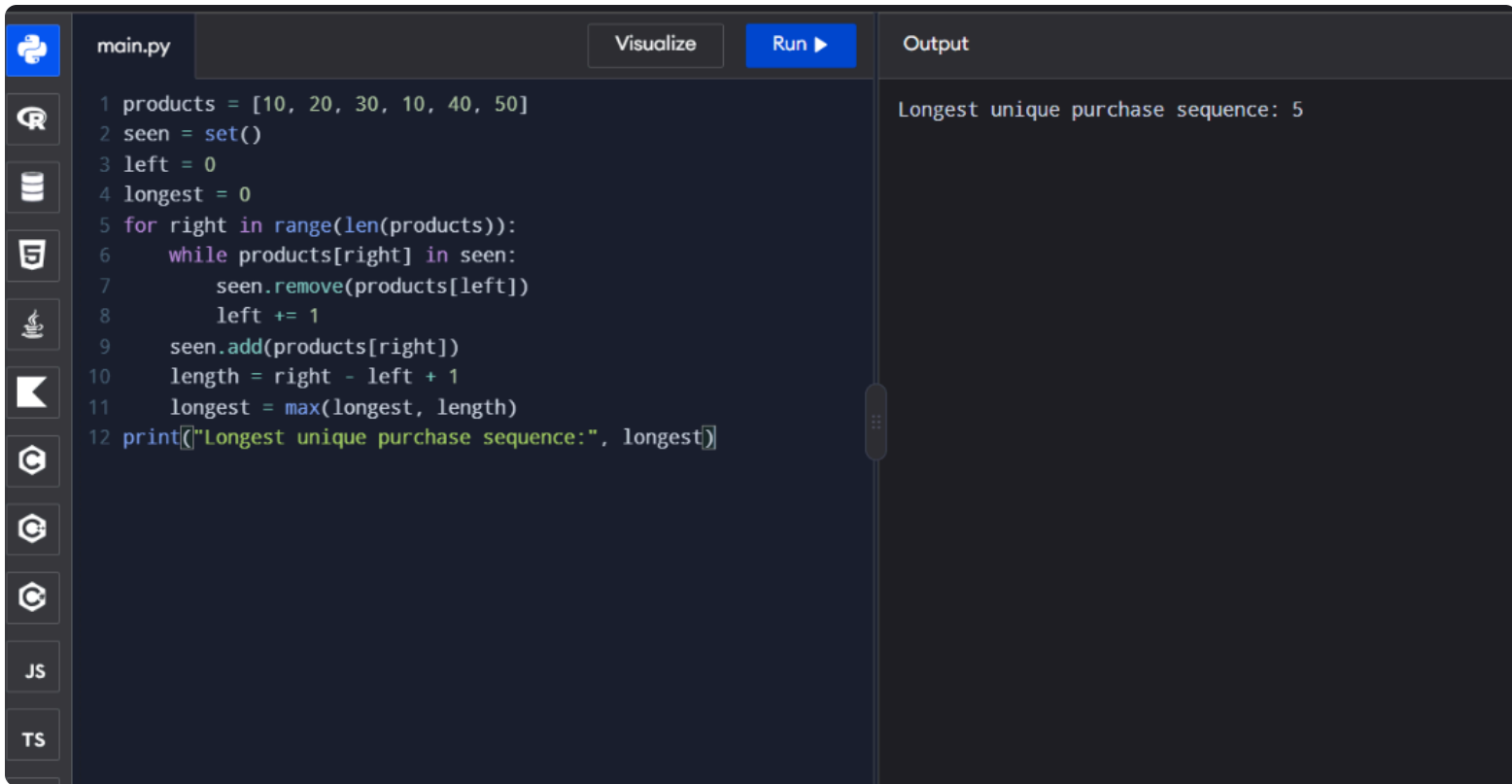
6. Customer Purchase History

An e-commerce application stores the product IDs purchased by a customer in chronological order. The same product may appear multiple times. The system needs to determine the longest sequence of consecutive purchases in which every product ID is unique.

Program

```
products = [10, 20, 30, 10, 40, 50]
seen = set()
left = 0
longest = 0
for right in range(len(products)):
    while products[right] in seen:
        seen.remove(products[left])
        left += 1
    seen.add(products[right])
    length = right - left + 1
    longest = max(longest, length)
print("Longest unique purchase sequence:", longest)
```

Output



```
main.py Visualize Run ▶ Output
1 products = [10, 20, 30, 10, 40, 50]
2 seen = set()
3 left = 0
4 longest = 0
5 for right in range(len(products)):
6     while products[right] in seen:
7         seen.remove(products[left])
8         left += 1
9     seen.add(products[right])
10    length = right - left + 1
11    longest = max(longest, length)
12 print("Longest unique purchase sequence:", longest)
```

Longest unique purchase sequence: 5

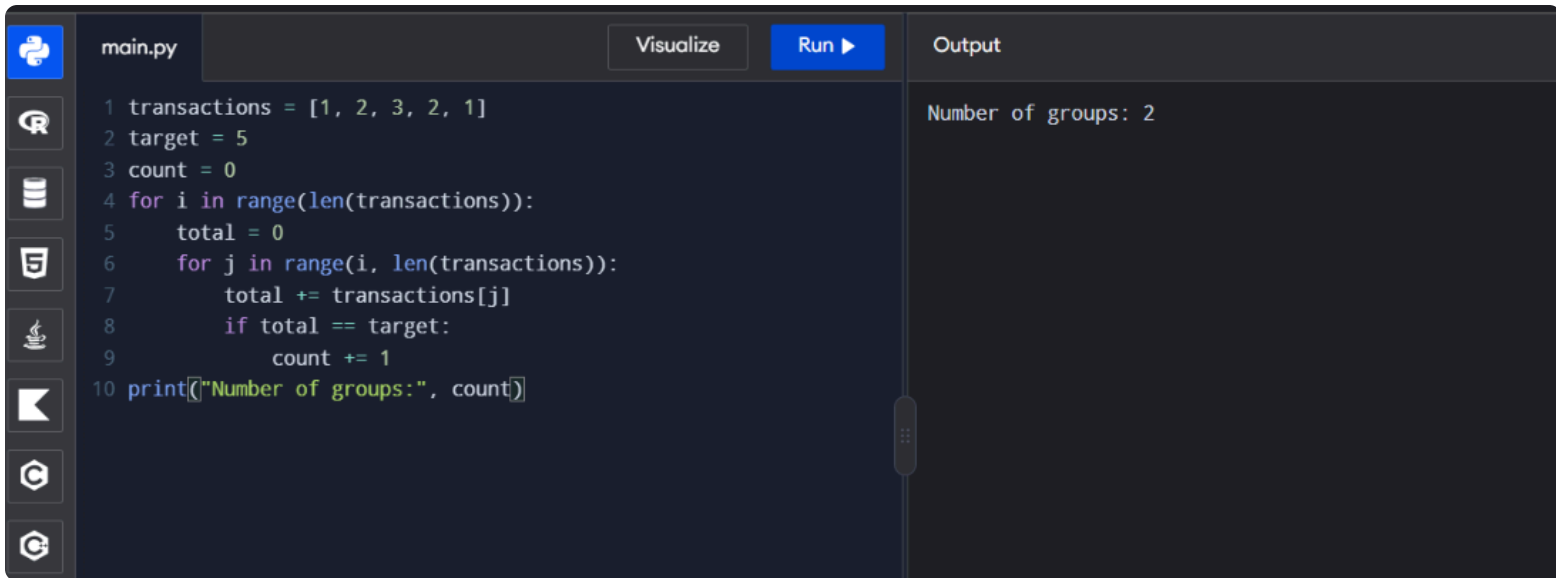
7. Bank Transaction Analysis

A bank stores transaction amounts for a customer's account. A continuous group of transactions may add up to a specific target amount. The auditing system needs to determine how many different continuous transaction groups produce exactly the specified amount.

Program

```
transactions = [1, 2, 3, 2, 1]
target = 5
count = 0
for i in range(len(transactions)):
    total = 0
    for j in range(i, len(transactions)):
        total += transactions[j]
        if total == target:
            count += 1
print("Number of groups:", count)
```

Output

A screenshot of a code editor interface. On the left is a sidebar with icons for file explorer, search, and other tools. The main area is split into two panes. The left pane, titled 'main.py', contains a Python script. The right pane, titled 'Output', shows the result of running the script. The script defines a list of transactions, a target value, and a count. It uses nested loops to find subarrays that sum to the target. The output shows that there are 2 such subarrays.

```
1 transactions = [1, 2, 3, 2, 1]
2 target = 5
3 count = 0
4 for i in range(len(transactions)):
5     total = 0
6     for j in range(i, len(transactions)):
7         total += transactions[j]
8         if total == target:
9             count += 1
10 print("Number of groups:", count)
```

Number of groups: 2

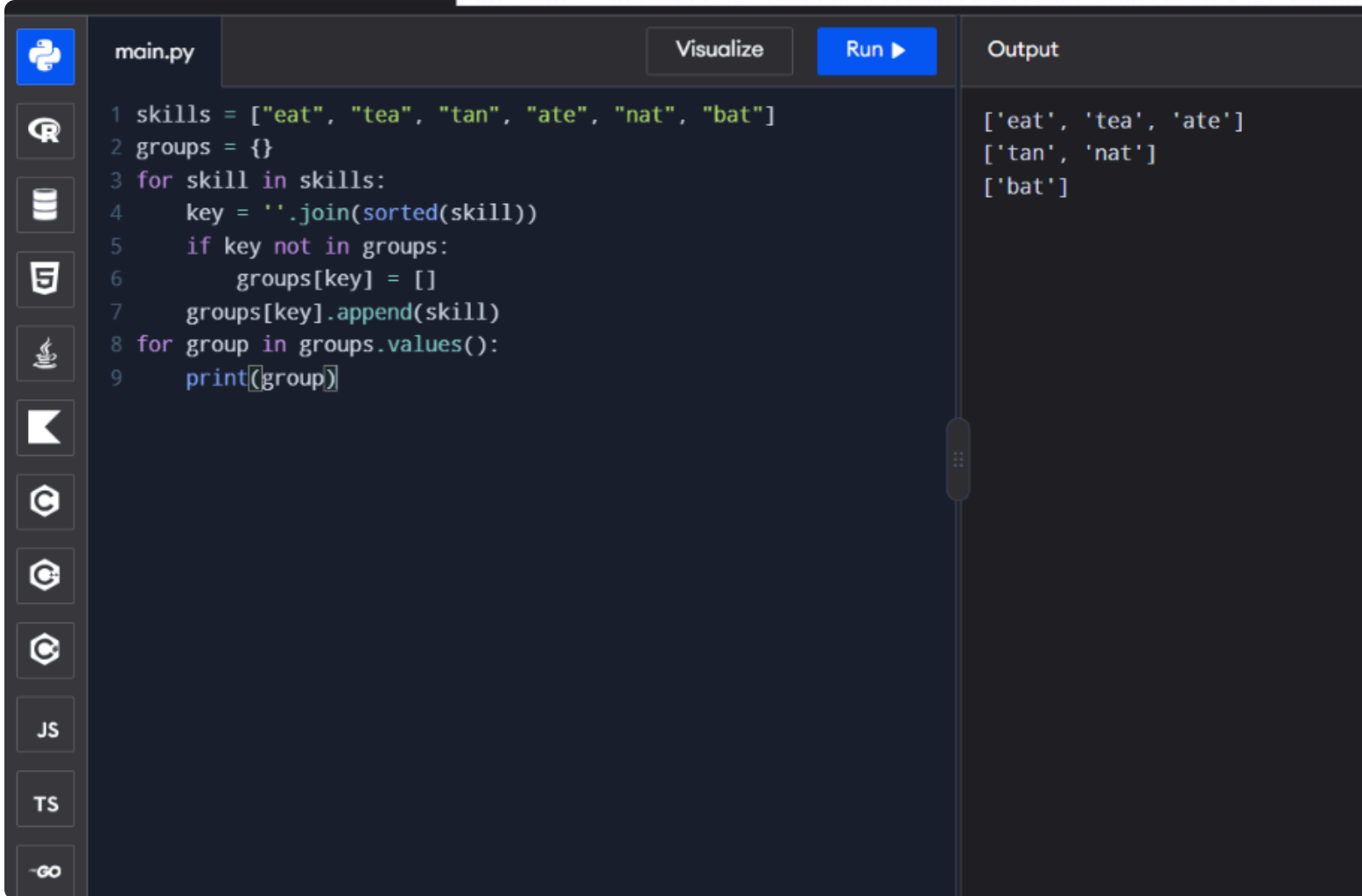
8. Employee Skill Grouping

A company receives a list of employee skill codes represented as strings. Employees having the same set of characters in their skill codes belong to the same skill category, even if the characters appear in a different order. The HR system needs to organize employees into appropriate skill groups.

Program

```
skills = ["eat", "tea", "tan", "ate", "nat", "bat"]
groups = {}
for skill in skills:
    key = ''.join(sorted(skill))
    if key not in groups:
        groups[key] = []
    groups[key].append(skill)
for group in groups.values():
    print(group)
```

Output



The screenshot shows a code editor interface with a dark theme. On the left, there is a sidebar with icons for Python, a search icon, a database icon, a file explorer icon, a terminal icon, a back icon, and a settings icon. Below these are buttons for 'JS' and 'TS', and a 'GO' button. The main area displays a Python script named 'main.py'. The script defines a list of skills, groups them into a dictionary based on sorted characters, and prints the groups. The output panel on the right shows the result of the script execution.

```
main.py Visualize Run ▶ Output
```

```
1 skills = ["eat", "tea", "tan", "ate", "nat", "bat"]
2 groups = {}
3 for skill in skills:
4     key = ''.join(sorted(skill))
5     if key not in groups:
6         groups[key] = []
7     groups[key].append(skill)
8 for group in groups.values():
9     print(group)
```

```
['eat', 'tea', 'ate']
['tan', 'nat']
['bat']
```

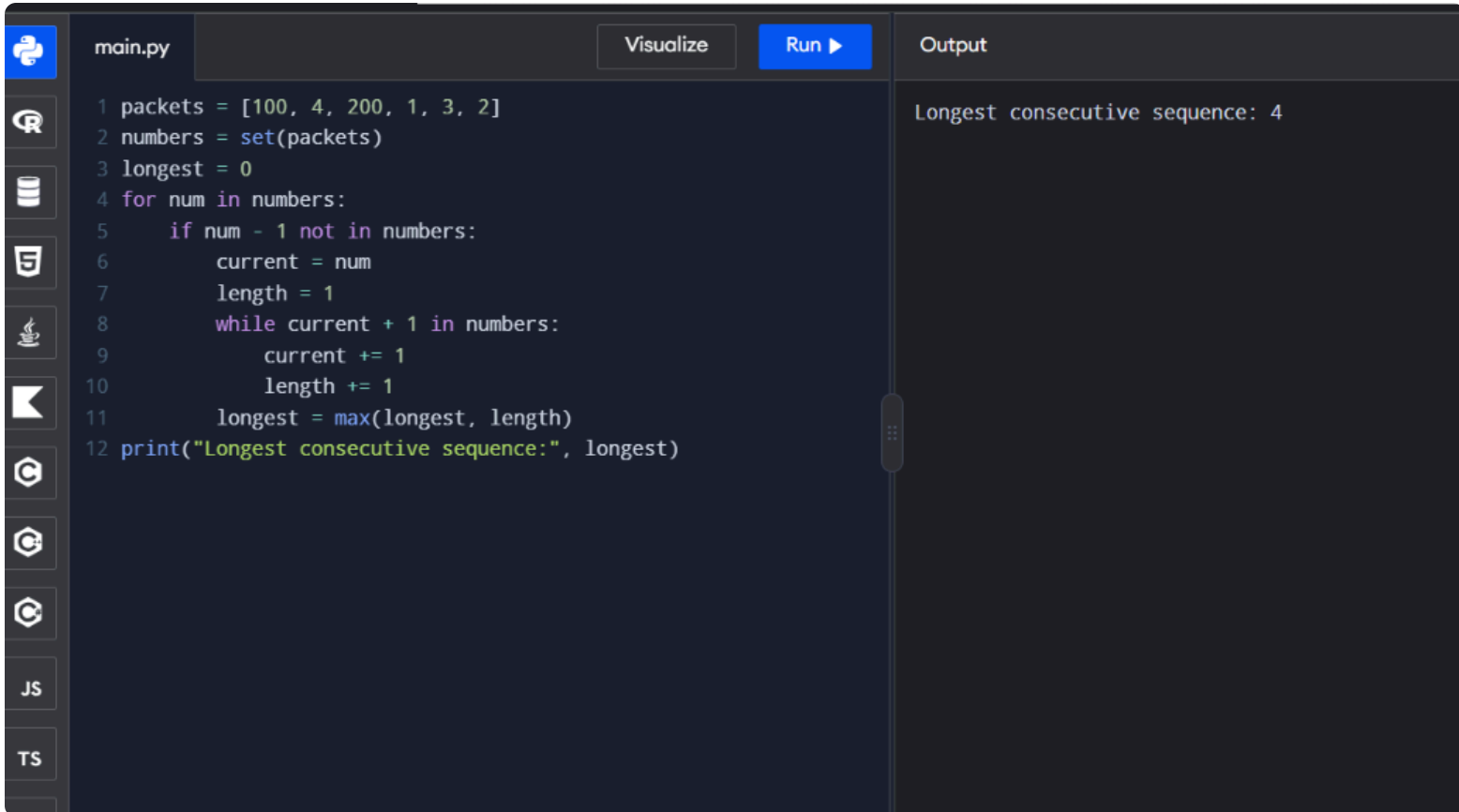
9. Network Packet Analysis

A network monitoring system receives packet identifiers in chronological order. The system must determine the longest sequence of consecutive packets whose identifiers form a continuous numerical sequence, regardless of their original order in the incoming data.

Program

```
packets = [100, 4, 200, 1, 3, 2]
numbers = set(packets)
longest = 0
for num in numbers:
    if num - 1 not in numbers:
        current = num
        length = 1
        while current + 1 in numbers:
            current += 1
            length += 1
        longest = max(longest, length)
print("Longest consecutive sequence:", longest)
```

Output



```
main.py Visualize Run ▶ Output

1 packets = [100, 4, 200, 1, 3, 2]
2 numbers = set(packets)
3 longest = 0
4 for num in numbers:
5     if num - 1 not in numbers:
6         current = num
7         length = 1
8         while current + 1 in numbers:
9             current += 1
10            length += 1
11            longest = max(longest, length)
12 print("Longest consecutive sequence:", longest)
```

Longest consecutive sequence: 4

10. Hospital Appointment Scheduling

A hospital receives appointment requests represented by starting and ending times. Some appointments overlap with each other. The scheduling system needs to combine overlapping appointment periods so that the final schedule contains only non-overlapping time ranges.

Program

```
appointments = [[1, 3], [2, 6], [8, 10], [9, 12]]
appointments.sort()
merged = []
for appointment in appointments:
    start = appointment[0]
    end = appointment[1]
    if not merged:
        merged.append([start, end])
    elif start <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])
print("Final schedule:", merged)
```

Output



JS

TS

main.py

Visualize

Run ▶

```
1 appointments = [[1, 3], [2, 6], [8, 10], [9, 12]]
2 appointments.sort()
3 merged = []
4 for appointment in appointments:
5     start = appointment[0]
6     end = appointment[1]
7     if not merged:
8         merged.append([start, end])
9     elif start <= merged[-1][1]:
10         merged[-1][1] = max(merged[-1][1], end)
11     else:
12         merged.append([start, end])
13 print("Final schedule:", merged)
```

Output

Final schedule: [[1, 6], [8, 12]]